

# VALIDATE Worksheet — 30 Minutes

---

You're going to write the test before you write the answer. This feels backwards — and that's the point. By deciding what a "good answer" looks like *before* you've seen the data, you have an objective standard to measure against later, instead of just eyeballing the result and deciding it "looks right."

## A few terms you'll see on this sheet:

- **Eval** — short for "evaluation." A question plus a checklist of things a good answer should include. Think of it as a rubric.
- **Assertion** — one item on that checklist. A specific, checkable statement like "the response names which table the data came from." Not vague ("the answer is good") — specific enough that anyone reading the response could say yes or no.
- **JSON** — a text format for structured data. It uses curly braces {}, square brackets [], and quoted strings "like this". You'll fill in a template — you don't need to know JSON to do it, just follow the pattern and be careful with commas and quotes.

**You have not queried the database yet, and that's correct.** You're writing a checklist based on what you already know about your domain — not based on a specific number you haven't seen. You'll discover the actual data in the next block.

Keep your Define worksheet's Question Spec open — you're writing this checklist for that exact question.

---

## 1. Recall the failure demo — 5 min

You watched an agent answer "how many employees do we have, I'm in a hurry" twice this morning — once with no skill, once with provenance-footer loaded.

Write down, in your own words, **two specific things** that made the first answer worse than the second. Not "it was less good" — specific. Did it state a source? A population? Did it hedge, or state a number with false confidence?

You're about to turn observations like these into assertions.

---

## 2. Pick one generalizable rule for your domain — 5 min

Every domain in this warehouse has a *pattern-level* risk that generalizes — you don't need today's specific number to know the risk exists.

| DOMAIN    | THE PATTERN TO GUARD AGAINST                                                                    |
|-----------|-------------------------------------------------------------------------------------------------|
| Attrition | A rate reported with no stated denominator, or a denominator choice buried instead of disclosed |

**Compensation** Values combined across currencies (or other units) without saying so

**Engagement** A trend compared across periods without confirming the instrument stayed the same

Pick the one matching your domain. This is scaffolding, not the answer — it tells you *what kind* of thing to check for, not what you'll actually find when you query the warehouse next block.

---

### 3. Write your checklist — 10 min

Write **2 to 3 things** a good answer to your question should include. Each one must be specific enough that a stranger reading the response could check it off — yes or no, not "sort of."

#### Examples of weak vs strong:

##### WEAK (CAN'T BE CHECKED)

"The answer is accurate"

"The analysis is thorough"

"It handles the data correctly"

##### STRONG (ANYONE CAN VERIFY)

"The response states which table the data came from"

"The response names which types of employees are counted"

"The response addresses whether the measurement stayed consistent across the years being compared"

At least one of your items should connect to the pattern from step 2 — for Attrition: *"the response states the denominator used to compute the rate."* For Compensation: *"the response states which currency the figures are in."* For Engagement: *"the response addresses whether the survey scale is consistent across the periods being compared."*

**Now put it in the template.** Open a text editor (Notepad, VS Code, or any editor that saves plain text), paste the template below, and fill in the blanks. Be careful with the punctuation — every comma, quote mark, and bracket matters:

```
{
  "skill_name": "<your domain, e.g. engagement>",
  "evals": [
    {
      "id": 1,
      "prompt": "<paste your question from Define, word for word>",
      "expected_output": "<one sentence describing what a good answer does>",
      "assertions": [
        "<your first checklist item>",
        "<your second checklist item>"
      ]
    }
  ]
}
```

### Important formatting rules:

- Every piece of text must be inside double quotes "like this" — not single quotes
- Items in a list are separated by commas, but there's **no comma after the last item** (this is the most common mistake)
- Save the file as `my-eval.json` inside the `evals` folder

To check if your file is valid, run this in your terminal:

**Windows:** `python -c "import json; json.load(open('evals/my-eval.json')); print('Valid!')"`

**Mac/Linux:** `python3 -c "import json; json.load(open('evals/my-eval.json')); print('Valid!')"`

If it prints "Valid!" you're good. If it shows an error, it will point to the exact line and character that's wrong — usually a missing comma or quote.

---

## 4. Add one negative consideration — 5 min

This pack keeps at least one negative test per skill — a case the skill should **not** change. Skipping this is how `provenance-footer` shipped a bug that attached a full footer to a simple schema lookup; it's in this repo's own history.

Write one line: what would an answer to your question look like if it were **overcautious** — padded with process, hedged into uselessness, or answering something nobody asked? You don't need to formalize this as a full eval today. Naming it is enough.

---

## 5. Predict — 5 min

Before anyone runs anything: for each assertion you wrote, guess whether a plain agent with **no skill loaded** would pass or fail it.

```
Assertion 1: [PASS / FAIL] — because ____
Assertion 2: [PASS / FAIL] — because ____
```

Write the guess down now. You'll find out if you were right during the ablation block at 2:15 — and being wrong is just as useful as being right. If you guessed PASS and it failed, that's a real gap you didn't know you had.

---

### Share-out — when the facilitator calls time

One sentence:

*"My strongest assertion is \_\_\_, and I predict a plain agent will \_\_\_ it."*

Keep this worksheet. In Build, once you've queried the warehouse and found your domain's actual Gotcha, come back and check: did the trap you found match the pattern you guarded against here? If it's different, that's fine — that's new information you write into the skill, not a sign you did this block wrong.